High-Level Design Document: `windows-gui` MCP Server — A Playwright-Style API for Agentic AI Control of the Windows Desktop

TL;DR

- **Build the server in C#/.NET 8 on top of FlaUI (a maintained UIA2/UIA3 wrapper), using Microsoft UI Automation (UIA) as the primary read/control mechanism and a lightweight OmniParser-v2-style CV/OCR fallback (finetuned YOLOv8 + Florence-2 0.23B + Windows.Media.Ocr) for non-accessible/custom-rendered apps and games.** This mirrors the architecture Microsoft Research validated in UFO2, whose hybrid UIA+vision detection beat both UIA-only and vision-only pipelines.
- **Expose a Playwright-MCP-style "snapshot + ref" API:** a `snapshot` tool returns a compact, structured accessibility tree where every interactive element gets a stable ref (e.g., `w1e43`); actions (`click`, `type_text`, `set_value`, `toggle`, `expand`, `scroll`, `drag`, `hotkey`) target refs rather than pixel coordinates, with auto-waiting and screenshots as a vision adjunct. This is exactly the model proven by shanselman/FlaUI-MCP and Playwright MCP.
- **The single biggest design decision — accessibility-first, vision-fallback — is correct and evidence-backed.** UIA gives semantic, DPI-independent, token-cheap element targeting; CV covers the minority of controls UIA cannot see. Treat synthetic input (UIA control patterns + SendInput) as primary actuation, reserve coordinate clicks for vision-detected elements, and design around UIPI/UAC elevation limits from day one.

Key Findings

1. **Accessibility-first is the proven winning architecture.** Microsoft's UFO2 ("The Desktop AgentOS", arXiv:2504.14603, Zhang et al.) fuses Windows UI Automation with OmniParser-v2 vision grounding into a "hybrid control detection" pipeline. Per the paper (§6.3): "The hybrid method consistently outperforms either UIA-only or OmniParser-only settings, raising the overall success rate and converting up to 9.86% of previously irrecoverable cases into completions." UFO2 with the o1 model reaches 30.5% success on Windows Agent Arena (WAA) and 32.7% on OSWorld-W; UFO2-base with GPT-4o reaches 23.4% on WAA, "outperforming the best existing baseline, Operator (20.8%), by 2.6%." On OSWorld-W, "UFO2 achieves a 28.6% SR compared to Operator's 14.3%, effectively doubling its success rate." The accessibility tree is text (~200–400 tokens/snapshot in Playwright MCP) versus thousands for screenshots — cheaper, faster, more deterministic.
2. **The MCP "snapshot + ref" pattern from Playwright is directly portable to Windows.** Playwright MCP returns a structured accessibility snapshot where each element has a `[ref=e5]` and tools like `browser_click {ref}` act on refs, not pixels. shanselman/FlaUI-MCP already ports this exactly to Windows desktop (`windows_snapshot`, `windows_click`, refs like `w1e43`) using FlaUI + UIA. This is the template to follow. `Playwright`

3. **The existing open-source Windows MCP landscape is fragmented between two philosophies.** Coordinate/vision-based + input-injection servers (CursorTouch/Windows-MCP, MCPControl, mcp-windows-desktop-automation/AutoIt, mukul975/mcp-windows-automation, winremote-mcp) vs accessibility-tree servers (shanselman/FlaUI-MCP, RandyNorthrup/win32-mcp-server, Sbroenne/WindowsMcp, CursorTouch/Windows-Use). The accessibility-tree camp is converging on the right answer; none yet combines a clean Playwright-style API with a robust integrated CV fallback — that is the gap this design fills.
4. **C#/.NET with FlaUI is the best-fit implementation stack.** UIA is a native COM API; .NET has first-class, strongly-typed bindings via System.Windows.Automation and the actively maintained FlaUI library (UIA2 + UIA3). Rust (uiautomation crate / windows-rs) and Python (comtypes/pywinauto/uiautomation) are viable but less mature for this specific job. .NET hits the sweet spot of native UIA access, performance, ecosystem, and the existence of FlaUI-MCP as a starting point.
5. **Lightweight CV fallback should be a layered pipeline, not a single model.** Recommended: Windows.Media.Ocr (built-in, free, on-device) or RapidOCR (ONNX, lowest memory) for text, plus an OmniParser-v2-style finetuned YOLOv8 icon detector + Florence-2 (0.23B) captioner for interactable-element detection. Per the microsoft/OmniParser-v2.0 model card, V2 runs at "0.6s/frame on A100, 0.8s on single 4090"; Microsoft Research states V2 "reduces the latency by 60% compared to the previous version" and that "Omniparser+GPT-4o achieves state-of-the-art average accuracy of 39.6 on … ScreenSpot Pro … a substantially improvement on GPT-4o's original score of 0.8."

Details

1. Landscape of existing tools, harnesses, and frameworks

Microsoft Research agent work (the most relevant prior art):

- **UFO / UFO2 / UFO3 (microsoft/UFO, MIT).** UFO (arXiv:2402.07939) was the first UI-focused dual-agent (HostAgent + AppAgent) framework for Windows, grounding GPT-Vision actions through a UIA-based control interaction module, with Win32 backend support. UFO2 (arXiv:2504.14603, April 2025) rebranded it a "Desktop AgentOS" with: a centralized HostAgent + application-specialized AppAgents; a **hybrid control detection pipeline** fusing UIA with OmniParser-v2 vision; a unified GUI–API "Puppeteer" action layer; a continuous knowledge substrate (RAG from help docs/Bing/demonstrations); speculative multi-action execution; and a Picture-in-Picture desktop. UFO3 "Galaxy" (Nov 2025) extends to multi-device orchestration and uses MCP servers to coordinate device agents. This is the canonical reference architecture for this project. ResearchGate + 2
- **OmniParser / OmniParser V2 (microsoft/OmniParser).** A pure-vision screen parser: a finetuned YOLOv8 interactable-region detector + a finetuned Florence-2 (0.23B; earlier BLIP-2) icon-caption model + OCR (PaddleOCR), producing a structured, "set-of-marks"–style list of elements with bounding boxes and functional captions. UFO2 §3.4 describes it as combining "a lightweight YOLO-v8 detector with a fine-tuned Florence-2 (0.23B) encoder." **Licensing split (critical):** the icon_detect (YOLO-derived) weights are **AGPL**, while icon_caption (Florence/BLIP-2) is MIT — material for a permissively licensed

product. OmniTool pairs OmniParser with a Windows 11 VM and arbitrary VLMs (GPT-4o/o1/o3-mini, DeepSeek R1, Qwen2.5-VL, Claude).

- **Power Automate Desktop** is Microsoft's production RPA tool; its UIPI troubleshooting docs are a goldmine of real-world elevation/permission failure modes (locked desktop, UAC dialogs, elevated targets, minimized RDP).

Frontier-lab "computer use" agents (vision-first, the contrast case):

- **Anthropic Computer Use** (anthropics/claude-quickstarts, computer-use-demo): a schema-less `computer` tool (screenshot + mouse/keyboard) driven in an agent loop, shipped as a Dockerized Linux+VNC reference. Pure pixel-coordinate approach; "you own the loop" (the integrator captures screenshots, maps coordinates, executes actions); current beta header `computer-use-2025-11-24`. Honest community assessments call it slow, expensive, and token-rate-limited for production. LaoZhang AI Blog
- **OpenAI CUA / Operator**: GPT-4o vision + RL, issues `click(x,y)`/`type(text)` from screenshots via the Responses API `computer_call`; browser-first, "without needing specialized APIs."
- Both deliberately avoid accessibility APIs — universal but fragile, expensive, and non-deterministic. This design takes the opposite, accessibility-first stance for native Windows, using vision only as fallback.

Existing open-source Windows MCP servers:

- **shanselman/FlaUI-MCP** (MIT, .NET 8) — closest to this design. Wraps FlaUI/UIA, builds agent-friendly accessibility snapshots, maps element refs ↔ AutomationElements, Playwright-inspired `windows_snapshot`/`windows_click`. "No screenshot parsing. No coordinate guessing." GitHub
- **CursorTouch/Windows-MCP** (MIT, Python) — UIAutomation + PyAutoGUI; "0.2 to 0.5 secs" action latency; tools `Click/Type/Scroll/Move/Shortcut/Wait/WaitFor/Screenshot/Snapshot`; a DOM mode for browsers; reached "2M+ Users in Claude Desktop Extensions"; explicitly "can't be used to play video games." GitHub
- **CursorTouch/Windows-Use** (MIT, Python) — an agent (not just a server) reading the UIA tree; `use_accessibility=True`, optional `use_vision`/`use_annotation`; IoU-based off-screen bbox trimming; caching to speed UIA3 tree traversal. GitHub
- **RandyNorthrup/win32-mcp-server** — 53 tools, UIA inspection + structured OCR with bounding boxes/confidence, fuzzy window matching, auto-DPI-awareness.
- **Sbroenne WindowsMcp** (.NET) — "automate by name, not pixels," DPI/theme/resolution-independent; tested against Win32, UWP, Electron (VS Code/Teams/Slack), Chromium.
- **MCPControl, mcp-desktop-automation (RobotJS), mcp-windows-desktop-automation (AutoIt), mukul975, winremote-mcp** — input-injection/screenshot servers; broad system control but coordinate-based and explicitly "experimental and potentially dangerous."

- **FlaUI** (.NET, open source) — modern UIA2+UIA3 wrapper; strongly typed; exposes native objects when needed; the recommended .NET choice; actively maintained.
- **pywinauto** (Python) — mature, dual Win32 + UIA backends; text-property-based; strong for Python shops.
- **Python-UIAutomation-for-Windows / uiautomation** (Python, comtypes) — direct IUIAutomation access; used by Windows-MCP.
- **uiautomation crate / windows-rs** (Rust) — `UIAutomation::new()`, matcher API, `send_keys`; windows-rs gives the complete projected Win32/UIA surface from win32metadata. Lib.rs
- **WinAppDriver** (Microsoft) — WebDriver/Appium protocol for Windows; reduced maintenance; Appium's Windows docs now point to the NovaWindows driver; known slowness/flakiness and root-access concerns. Avoid for new builds.
- **Winium** (deprecated), **White/TestStack.White** (legacy), **AutoIt/AutoHotkey** (scripting/input injection), **SikuliX/AirTest** (pure image-matching, good for Citrix/games but brittle to visual change), **Robot Framework** (orchestration).
- **Inspect.exe** (Windows SDK, legacy) and **Accessibility Insights for Windows** (open source, WPF, built on UIA) — essential developer tooling for examining the UIA tree, properties, patterns, and events during development.

2. Windows system APIs for GUI inspection and control

High-level — Microsoft UI Automation (UIA), the primary stack. UIA (UIAutomationClient/UIAutomationCore; COM `IUIAutomation`) exposes every UI element as an `IUIAutomationElement` in a tree rooted at the desktop, with control views and content views. Key concepts to map into the API:

- **Control types** (~38: Button, CheckBox, ComboBox, Edit, DataGrid, Document, Hyperlink, Image, List, ListItem, Menu, Tree, Window, etc.).
- **Control patterns** (~22): `Invoke` (click/press), `Value` (get/set text), `Toggle`, `ExpandCollapse`, `Scroll`/`ScrollItem`, `Selection`/`SelectionItem`, `Text` (rich text ranges), `Grid`/`GridItem`/`Table`, `Window`, `Transform` (move/resize), `RangeValue`, `Dock`. Clients query supported patterns then call methods (e.g., `IUIAutomationInvokePattern::Invoke`, `IUIAutomationScrollPattern::SetScrollPercent`).
- **Properties**: common element properties (Name, AutomationId, ClassName, ControlType, BoundingRectangle, IsEnabled, IsOffscreen, AcceleratorKey) plus pattern-specific properties; each has a PROPERTYID.
- **Conditions & tree walking**: `PropertyCondition`, `AndCondition`, `FindFirst`/`FindAll`, `TreeWalker` (GetParent/FirstChild/NextSibling) — the basis for locators and XPath-like queries.
- **MSAA / IAccessible** is the older predecessor; some apps (e.g., Firefox via IAccessible2) only expose accessibility through it, so a fallback bridge is valuable.

- **Performance is dominated by cross-process COM calls.** The critical optimization is **caching / bulk-fetch** via `IUIAutomationCacheRequest` (AddProperty/AddPattern, TreeScope, TreeFilter, `FindFirstBuildCache`/`GetUpdatedCache`): retrieve all needed properties/patterns for a whole subtree in one cross-process call, then read from the cache. Set `AutomationElementMode.None` when you only need data, not live actions. (Caching benefits are largest for WPF/custom server-side providers; less for Win32 client-side providers.) This is essential to make snapshots fast. Microsoft Learn

Low-level — Win32 windowing, input, and capture.

- **Window management**: `EnumWindows`, `FindWindow`, `GetWindowText`, `GetWindowRect`, `GetForegroundWindow`, `SetForegroundWindow`, `AttachThreadInput` (for foreground-stealing tricks).
- **Input injection**: `SendInput` is the Microsoft-recommended synthetic input API (superseding `keybd_event`/`mouse_event`); it injects into the input stream of the **foreground** window — you cannot target a specific background window, and it mixes with real user input. `SetCursorPos` for mouse positioning.
- **Message-based control**: `SendMessage`/`PostMessage` with `WM_GETTEXT`, `BM_CLICK`, etc. Per Raymond Chen ("The Old New Thing", 2025), **you cannot reliably simulate keyboard input with PostMessage** — many apps (Notepad, Word, WordPad) discard posted input; behavior is app-specific and unreliable. Message-based control is brittle and should not be primary.
- **Hooks**: `SetWindowsHookEx` for global hooks (rarely needed here).
- **Screen capture**: `BitBlt`/GDI (slow, CPU-heavy, simple, captures most windows); **Windows.Graphics.Capture (WGC)** (modern, GPU-shared-texture, cross-GPU, no injection, yellow capture border, Win10 1803+/11); **DXGI Desktop Duplication** (`IDXGIOutputDuplication::AcquireNextFrame`, fastest, handles fullscreen-exclusive DX apps/games, returns dirty rects, requires same-GPU as the display); **Magnification API** as a fallback. For agentic capture: prefer WGC for per-window capture; DXGI Desktop Duplication for full-screen/game capture.

Trade-offs & cross-cutting OS constraints:

- **Accessibility-tree control (UIA patterns)** is the most reliable and semantic: it invokes the control's own handlers regardless of foreground state, is DPI-independent (returns physical coordinates), and degrades gracefully. **Synthetic input (SendInput)** is needed for elements without actionable patterns and for vision-detected targets, but requires the target to be foreground/focused. **Message-based** is the least reliable.
- **UIPI / UAC / integrity levels**: A process can only send input/messages to processes at the **same or lower integrity level**. To drive elevated apps the server itself must run elevated (and even then UIPI plus the `uiAccess=true` manifest may be required to interact with secure UI). Locked desktop, open UAC/secure-desktop dialogs, screensavers, and minimized RDP all break automation (well-documented in Power Automate Desktop's UIPI troubleshooting).

- **DPI & multi-monitor**: The server must declare **Per-Monitor-V2 DPI awareness** so the OS does not virtualize coordinates. Even then, some apps (e.g., MS Teams, older Notepad) report inconsistent `BoundingRectangle`/`ElementFromPoint` values across mixed-DPI multi-monitor setups — a known, app-specific hazard requiring coordinate-reconciliation logic and validation against the captured frame.

Implementation language recommendation (researched): C#/.NET 8 + FlaUI (UIA3, with UIA2 fallback), P/Invoking Win32 for input/capture.

- *Why .NET over alternatives*: UIA is COM-native and .NET has the most mature, strongly-typed, well-documented bindings (System.Windows.Automation + FlaUI). FlaUI already wraps Invoke/Value/Toggle/Selection patterns, tree walking, and caching, and shanselman/FlaUI-MCP proves the MCP mapping. The official C# MCP SDK (ModelContextProtocol) exists. .NET gives near-native performance, easy P/Invoke to SendInput/DXGI/WGC, and trivial ONNX Runtime integration for the CV fallback.
- *Rust (windows-rs + uiautomation crate)*: excellent performance and a complete projected API surface, single static binary, but smaller ecosystem, more COM boilerplate, and no equivalent of FlaUI's batteries-included pattern wrappers — more build cost for marginal benefit.
- *Python (comtypes/pywinauto/uiautomation)*: fastest to prototype and the basis of Windows-MCP/Windows-Use, but COM marshalling overhead and the GIL hurt for large/fast tree snapshots, and packaging a CV stack is heavier.
- *C++*: maximum control/perf, highest development cost; reserve for a thin native capture/input helper if needed.

3. Lightweight CV / OCR fallback

The fallback fires only when UIA coverage is insufficient (empty/sparse tree, custom-rendered canvas/Electron/game UIs, or an action on a UIA element fails). Recommended layered pipeline, all on-device:

- **OCR (text + bounding boxes)**:
  - **Windows.Media.Ocr** — built into Windows, free, no dependencies, on-device; the natural default on the target OS.
  - **RapidOCR** (ONNX Runtime, Apache-2.0) — lowest memory, sub-second CPU inference, C#/C++/Java/Python bindings, PaddleOCR models converted to ONNX; best portable open-source option.
  - **PaddleOCR** (structured/multilingual, heavier, GPU-preferred), **Tesseract** (mature, ~92% on clean English in a 2024 benchmark, weak on complex layouts), **EasyOCR** (slower, ~500MB). Use these only for special cases.
- **Interactable-element detection (icons/buttons/fields)**: an **OmniParser-v2-style** pipeline — finetuned **YOLOv8** interactable-region detector + **Florence-2 (0.23B)** caption model — producing set-of-marks elements with bounding boxes and functional captions. Per the

model card, OmniParser V2 runs "0.6s/frame on A100, 0.8s on single 4090," "reduces the latency by 60% compared to the previous version," and (with GPT-4o) raised ScreenSpot-Pro grounding "from GPT-4o's original score of 0.8" to "39.6 average accuracy." **Licensing caveat**: OmniParser's icon_detect weights are AGPL; for a permissive product, either isolate them as an optional plugin or retrain/replace the detector. Replicate

- **GUI grounding models (optional, for natural-language→coordinate)**: UGround (UGround-V1 leads ScreenSpot), OS-Atlas, SeeClick, CogAgent, Ferret-UI, ScreenAI. All degrade sharply on tiny, high-resolution targets (ScreenSpot-Pro), so prefer detection+OCR+set-of-marks over end-to-end grounding for reliability on professional desktop apps.
- **Capture feeding the pipeline**: WGC per-window (GPU texture) or DXGI Desktop Duplication for games; downscale to ~1024px (Anthropic's XGA/1024×768 guidance) before model inference to bound latency/tokens.

4. Proposed `windows-gui` MCP server design

Architecture (layers):

1. **MCP transport layer** — JSON-RPC 2.0 over stdio (local default) and Streamable HTTP/SSE (remote, HTTPS-required per spec); built on the C# MCP SDK; tool discovery via `tools/list`.
2. **Tool/dispatch layer** — implements the tool handlers below; manages sessions, element-ref tables, and waits.
3. **Unified Element Model (UEM)** — the core abstraction: a normalized element record `{ref, source: "uia"|"cv", role/controlType, name, automationId, className, value, bbox(physical px), patterns[], states[], interactable}` merged from UIA and CV. Refs are stable within a snapshot (`w<window>e<n>`), mapping back to live `AutomationElement`s or CV detections.
4. **Perception layer** — *UIA primary*: cached tree walk (`IUIAutomationCacheRequest`) producing the snapshot; *CV fallback*: capture → OCR + YOLOv8/Florence-2 → detections; *fusion*: IoU-based dedup into one control graph.
5. **Action layer ("Puppeteer")** — prefers UIA control patterns (Invoke/Value/Toggle/etc.); falls back to SendInput at element center (focus window first) for CV/pattern-less elements; native hotkeys via SendInput.
6. **Capture layer** — WGC/DXGI/BitBlt.
7. **Cross-cutting** — DPI/monitor manager, security/elevation manager, auto-wait/retry engine, observability/trajectory logger.

Playwright → Windows concept mapping:

Playwright	`windows-gui` equivalent
Browser / Page	Application / Window (HWND + top-level AutomationElement)
`browser_snapshot` (a11y tree + refs)	`snapshot` (UIA tree + CV-merged refs)
`getByRole`	locate by `controlType`
`getByText` / `getByLabel`	locate by `name`/OCR text
CSS/XPath selector	UIA `automationId`/`className` + XPath-like tree query
`locator.click()`	`click {ref}` → InvokePattern or SendInput
`fill()` / `type()`	`set_value {ref}` (ValuePattern) / `type_text` (SendInput)
auto-waiting	auto-wait engine (poll UIA conditions/events)
`expect(...).toBeVisible()`	`wait_for {condition}`
`page.screenshot()`	`screenshot` (WGC/DXGI)
Vision mode (`--caps=vision`)	CV fallback mode

Proposed MCP tools (names, key params, returns):

- `list_windows {filter?}` → `[{handle, title, processName, pid, bbox, isForeground}]`
- `launch_app {path|appUserModelId, args?, waitForWindow?}` → `{handle, ...}`
- `attach {handleOrTitleRegex}` / `focus_window {handle}` → `{handle, focused}`
- `snapshot {handle, scope?: "window"|"focused-control", includeOffscreen?, useVision?: bool, detail?: "control"|"content"}` → structured tree of UEM nodes with refs (text-first, ~hundreds of tokens), optional screenshot
- `find_element {handle, by: "name"|"role"|"automationId"|"className"|"text"|"xpath", value, nth?}` → `{ref, ...}` or list
- `get_text {ref}` → `{text}` (TextPattern/ValuePattern, OCR fallback)
- `get_properties {ref}` → full UEM record incl. supported patterns + bbox
- `click {ref|point, button?: "left"|"right"|"middle", clickCount?}` → `{ok, snapshotDelta?}`
- `type_text {ref?, text, clearFirst?}` → `{ok}` (focus + ValuePattern or SendInput)
- `set_value {ref, value}` → `{ok}` (ValuePattern)
- `toggle {ref}` / `expand {ref}` / `collapse {ref}` / `select {ref}` → `{ok, newState}`
- `scroll {ref|point, direction, amount}` → `{ok}` (ScrollPattern or wheel)
- `drag {fromRef|fromPoint, toRef|toPoint}` → `{ok}`
- `hotkey {keys: "ctrl+s"}` → `{ok}` (SendInput)

- `wait_for {handle, condition: "element-exists"|"text-visible"|"window-active"|"element-enabled", target, timeoutMs}` → `{ok, ref?}`
- `screenshot {handle?|region?, scale?}` → `{imageBase64, bbox}`
- `inspect_at {point}` → UEM node under a point (ElementFromPoint, for debugging/grounding)

Most action tools optionally return a fresh snapshot delta (Playwright-style auto-snapshot), with an `includeSnapshot:false` switch for token economy.

Hybrid UIA+CV decision logic:

1. Always attempt the UIA snapshot first (cached bulk-fetch).
2. Trigger CV when: the UIA tree is empty/sparse for a visible window; the target window class is a known custom renderer (game engine, canvas, certain Electron/Qt apps); an action on a UIA element fails or the requested element isn't found; or the caller passes `useVision:true`.
3. Fuse: run capture → OCR + icon detection; convert detections to UEM nodes; **dedup by IoU against UIA bboxes** — following UFO2's rule that "Visual detections with Intersection-over-Union (IoU) greater than 10% against any UIA-derived control are discarded." (In UFO2's evaluation, "27.9% and 56.7% of OmniParser-v2 detections are discarded due to overlap with UIA" on the WAA and OSWorld-W benchmarks respectively.) Assign refs from a shared namespace so the agent sees one unified element list regardless of source.
4. Act: UIA-sourced refs → control patterns; CV-sourced refs → SendInput at bbox center after focusing the window and reconciling coordinates against current DPI/monitor.

Cross-cutting concerns:

- **Performance/latency**: aggressive UIA caching; snapshot scoping (window/subtree, control-view filter); lazy CV (only on miss); downscaled capture; optional out-of-context snapshot persistence (Playwright's CLI disk approach cut a representative task from ~114k to ~27k tokens, ~4x). Target sub-second perception per step like Windows-MCP's 0.2–0.5s action latency.
- **DPI/multi-monitor**: Per-Monitor-V2 manifest; store all bboxes in physical pixels; reconcile UIA `BoundingRectangle` against the captured frame; special-case known buggy apps.
- **Security/permissions**: run un-elevated by default; document that driving elevated apps needs an elevated server (and possibly `uiAccess=true`); surface UIPI/locked-desktop/UAC failures as structured, actionable errors; gate destructive actions behind client confirmation (human-in-the-loop), echoing Anthropic's prompt-injection screenshot classifier and the "experimental and potentially dangerous" warnings on existing servers.
- **Reliability/auto-wait/retry**: poll UIA conditions and subscribe to UIA events (FocusChanged/property-changed) with bounded timeouts and exponential backoff; verify post-action state (e.g., toggle reflected) before returning.

- **Error handling**: typed errors — `ElementNotFound`, `Ambiguous`, `PatternNotSupported`, `UIPIBlocked`, `Timeout`, `WindowGone`, `CVLowConfidence` — so the agent can adapt (e.g., retry via vision).
- **Observability**: trajectory logging (snapshots, tool calls, before/after screenshots), à la UFO2 logs and Anthropic best-practices, for replay/debugging.

Recommendations

Staged build plan:

1. **MVP (UIA-only, Playwright-style core).** Fork/learn from shanselman/FlaUI-MCP. Ship `list_windows`, `launch_app`, `attach`, `focus_window`, `snapshot` (cached UIA, text-first), `find_element`, `click`, `type_text`, `set_value`, `get_text`, `screenshot`, `wait_for`. Declare Per-Monitor-V2 DPI. **Benchmark gate:** reliable end-to-end on Calculator, Notepad, File Explorer, Settings, and a WinForms and a WPF app.
2. **Robustness.** Add toggle/expand/scroll/drag/hotkey via control patterns; auto-wait engine + UIA event subscriptions; typed errors; trajectory logging; caching tuned via `CacheRequest`. **Gate:** stable multi-step office workflows; measure snapshot latency and token size.
3. **CV fallback.** Integrate WGC/DXGI capture + Windows.Media.Ocr/RapidOCR + an OmniParser-v2-style detector via ONNX Runtime; implement IoU fusion (>10% discard) + unified refs + `useVision`. **Gate:** can locate/click a control in a canvas/game window where UIA is blind; measure added latency (target <~1s/frame; note this is GPU-class — CPU-only will be slower).
4. **Security & elevation hardening.** Elevated-mode docs + `uiAccess` manifest path; human-in-the-loop confirmation for destructive actions; UIPI error surfacing.
5. **Evaluate against Windows Agent Arena** to quantify success-rate, using UFO2 (27.9% WAA with GPT-4o, 30.5% with o1) as the north-star comparison.

Decision thresholds that would change the plan:

- If a large fraction of target apps are custom-rendered/games (UIA-blind), **promote CV from fallback to co-primary** and invest in a permissively licensed detector (the AGPL icon_detect weights would otherwise block commercial use).
- If the deployment is Python-centric or must share code with Windows-Use/Windows-MCP, accept the Python stack despite COM/GIL overhead.
- If sub-200ms perception is required at scale, consider a Rust (windows-rs) capture/snapshot core behind the .NET service.

Caveats

- **Benchmark numbers are research-context, not production guarantees.** UFO2's 23.4–30.5% WAA success and OmniParser's ScreenSpot-Pro 39.6 reflect specific harnesses/models; absolute agentic success on arbitrary apps remains modest industry-wide. Treat them as relative evidence for the architecture, not throughput promises. (Note:

secondary blogs sometimes misattribute UFO2's "25%+ recovery" figure to hybrid detection; per the paper, hybrid detection recovers 9.86% of irrecoverable cases, while the ~25% recovery figure refers to GPT-4o's control-detection-failure recovery under the GUI+API configuration.)

- **UIA coverage is uneven.** Some apps expose poor/no accessibility trees (the original UFO paper notes many Adobe Acrobat control types aren't UIA-supported); Electron/Qt/Java/game UIs vary widely. The CV fallback is essential, not optional, for general coverage.
- **Elevation and UIPI are hard limits, not bugs.** No design can drive an elevated app from a non-elevated process, or interact with the secure desktop/UAC prompt, without appropriate elevation/`uiAccess` — plan operationally around this.
- **Licensing split in OmniParser** (AGPL detector vs MIT captioner) must be resolved before any commercial distribution.
- **Coordinate reconciliation across mixed-DPI multi-monitor setups is genuinely buggy in some apps** (Teams, older Notepad); always validate UIA bboxes against the captured frame before coordinate-based clicks.
- **Security posture**: giving an AI agent GUI control is high-risk; existing servers carry explicit "experimental and potentially dangerous" disclaimers, and the broader CUA literature flags screenshots as the trust anchor for confused-deputy/prompt-injection attacks. Sandboxing (VM/Windows Sandbox), human-in-the-loop, and prompt-injection defenses are mandatory for any non-trivial deployment.
- **CV/agent latency figures are vendor/research numbers on specific GPU hardware** (A100/RTX 4090); real-world CPU-only latency on the fallback path will be substantially higher.

References

- UFO: A UI-Focused Agent for Windows OS Interaction — arXiv:2402.07939; github.com/microsoft/UFO; Microsoft Research publication page.
- UFO2: The Desktop AgentOS — arXiv:2504.14603 (Zhang et al.); microsoft.github.io/UFO (UFO² overview, hybrid control detection, Puppeteer action layer).
- OmniParser for Pure Vision Based GUI Agent — arXiv:2408.00203; github.com/microsoft/OmniParser; Microsoft Research "OmniParser" and "OmniParser V2: Turning Any LLM into a Computer Use Agent"; huggingface.co/microsoft/OmniParser-v2.0; replicate.com/microsoft/omniparser-v2.
- Microsoft Learn UI Automation docs: UI Automation Overview; Control Patterns Overview; Control Types Overview; Properties Overview; UI Automation Specification; Caching UI Automation Properties (IUIAutomationCacheRequest) / Caching in UI Automation Clients; Inspect.exe; Accessibility testing.
- Power Automate Desktop — "UIPI Issues with UI and Browser Automation Actions" (Microsoft Learn).

- "The Old New Thing" — "You can't simulate keyboard input with PostMessage, revisited" (devblogs.microsoft.com, 2025).
- DXGI Desktop Duplication API (Microsoft Learn); Windows.Graphics.Capture; D3DShot.
- Playwright MCP — playwright.dev/mcp (snapshots, interaction tools); github.com/microsoft/playwright-mcp; Playwright CLI token comparison.
- shanselman/FlaUI-MCP; CursorTouch/Windows-MCP and Windows-Use; RandyNorthrup/win32-mcp-server; Sbroenne WindowsMcp; MCPControl; mcp-windows-desktop-automation.
- Anthropic Computer Use — platform.claude.com computer-use-tool docs; github.com/anthropics/claude-quickstarts (computer-use-demo). OpenAI — "Computer-Using Agent" and "Introducing Operator."
- FlaUI (github.com/FlaUI/FlaUI); pywinauto; uiautomation crate (leexgone/uiautomation-rs) and Rust for Windows (windows crate, Microsoft Learn); WinAppDriver; Accessibility Insights for Windows.
- OCR: RapidOCR (RapidAI); PaddleOCR; Tesseract; Windows.Media.Ocr; OCR benchmarks (Nanonets, Modal).
- GUI grounding: UGround — arXiv:2410.05243; OS-Atlas — arXiv:2410.23218; SeeClick; ScreenSpot-Pro — arXiv:2504.07981.
- Model Context Protocol specification — modelcontextprotocol.io (Tools; JSON-RPC 2.0 transport).
- CUA safety literature — "A Survey on the Safety and Security Threats of Computer-Using Agents" (arXiv:2505.10924); "Visual Confused Deputy" (arXiv).